Name: Mohammad Abaid

Roll no: B-08

Branch: AI&DS(B)

Experiment:08

AIM:implement and analyze various methods of studying correlation and regression including Karl Pearson's coefficient and simple linear regression.

THOERY:

1. What is correlation? How is it different from regression?

Correlation measures the strength and direction of a linear relationship between two or more variables. It tells us *how strongly* two variables are related and in *what direction* (positive or negative). Correlation values range from -1 to +1, where:

- +1 indicates a perfect positive linear relationship.
- -1 indicates a perfect negative linear relationship.
- 0 indicates no linear relationship.

Regression, on the other hand, is a statistical method used to model the relationship between a dependent variable and one or more independent variables. Its primary goal is to predict the value of the dependent variable based on the independent variable(s). Regression attempts to find a mathematical equation that best describes the relationship, allowing for prediction and understanding of cause-and-effect (or predictive) relationships.

Key Differences:

Feature	Correlation	Regression
Purpose	Measures strength and direction of association.	Models relationship for prediction and explanation.
Variables	Treats variables symmetrically (no dependent/independent distinction).	Distinguishes between dependent (response) and independent (predictor) variables.
Output	A single value (correlation coefficient) between -1 and +1.	An equation (e.g., y = mx + c) that predicts the dependent variable.
Causation	Does not imply causation.	Can imply causation (if conditions are met) or predictive power.
Goal	Describe the relationship.	Predict or estimate values.

2. What is simple linear regression? Explain with suitable example.

Simple linear regression is a statistical model that estimates the relationship between one independent variable (predictor) and one dependent variable (response) using a straight line. The goal is to find the best-fitting straight line through the data points that minimizes the sum of the squared differences between the observed values and the values predicted by the line.

The equation for a simple linear regression model is typically written as:

$Y = \beta_0 + \beta_1 X + \varepsilon$

Where:

- Y is the dependent variable (the one we want to predict).
- X is the independent variable (the predictor).
- β_0 (beta-naught) is the Y-intercept (the value of Y when X is 0).
- β_1 (beta-one) is the slope of the line (the change in Y for a one-unit change in X).
- ε (epsilon) is the error term, representing the difference between the actual and predicted values.

Example: Predicting Exam Scores based on Study Hours

Imagine a teacher wants to predict a student's final exam score based on the number of hours they spent studying. Here:

- **Dependent Variable (Y):** Final Exam Score
- **Independent Variable (X):** Hours Studied

The teacher collects data from several students:

Hours Studied (X)	Exam Score (Y)
2	60
4	70
5	75
6	80
8	90

Using simple linear regression, the teacher would find the β_0 and β_1 that best fit this data. Let's say the regression analysis yields the following equation:

$$\text{Exam Score} = 50 + 5 * \text{Hours Studied}$$

- $\beta_0 = 50$: This means if a student studies 0 hours, the predicted exam score is 50.
- $\beta_1 = 5$: This means for every additional hour studied, the predicted exam score increases by 5 points.

With this model, if a new student studies 7 hours, the teacher can predict their score:

$$\text{Predicted Exam Score} = 50 + 5 * 7 = 50 + 35 = 85$$

This example illustrates how simple linear regression helps in understanding the relationship between two variables and making predictions.

3. How do you calculate Pearson correlation in Python?

Pearson correlation coefficient (often denoted as `r`) can be calculated in Python using the `pandas` library or the `scipy.stats` module. Both are widely used and straightforward.

Using `pandas.DataFrame.corr()`: This is convenient when you have your data in a pandas DataFrame and want to calculate correlations between multiple columns or specific pairs.

Using `scipy.stats.pearsonr()`: This function returns both the Pearson correlation coefficient and the 2-tailed p-value. It's useful when you need statistical significance along with the correlation value for two specific arrays.

```
import pandas as pd
import numpy as np
from scipy.stats import pearsonr

# Sample data
data = {
    'Hours Studied': [2, 4, 5, 6, 8, 3, 7, 1, 9, 5],
    'Exam Score': [60, 70, 75, 80, 90, 65, 85, 55, 95, 72],
    'Attendance': [8, 9, 7, 10, 9, 6, 8, 5, 10, 7]
}
df = pd.DataFrame(data)

print("DataFrame:")
display(df.head())

# --- Method 1: Using pandas.DataFrame.corr() ---
print("\nPearson Correlation Matrix (using pandas):")
# By default, .corr() calculates Pearson correlation
print(df.corr(method='pearson'))

# To get correlation between two specific columns from DataFrame
print("\nPearson Correlation between 'Hours Studied' and 'Exam Score'
(using pandas):")
print(df['Hours Studied'].corr(df['Exam Score']))

# --- Method 2: Using scipy.stats.pearsonr() ---
# This returns the correlation coefficient and the p-value
correlation_coefficient, p_value = pearsonr(df['Hours Studied'],
df['Exam Score'])

print("\nPearson Correlation Coefficient (using
scipy.stats.pearsonr):", correlation_coefficient)
print("P-value (using scipy.stats.pearsonr):", p_value)

# You can also manually calculate it for understanding
x = df['Hours Studied']
y = df['Exam Score']

n = len(x)
sum_xy = np.sum(x * y)
sum_x = np.sum(x)
sum_y = np.sum(y)
sum_x2 = np.sum(x**2)
```

```
sum_y2 = np.sum(y**2)

numerator = n * sum_xy - sum_x * sum_y
denominator = np.sqrt((n * sum_x2 - sum_x**2) * (n * sum_y2 -
sum_y**2))

pearson_r_manual = numerator / denominator
print("\nPearson Correlation Coefficient (manual calculation):",
pearson_r_manual)
```

DataFrame:


```
{"summary":"{\n  \"name\": \"print(\\\"\\\\nPearson Correlation
Coefficient (manual calculation):\\\", pearson_r_manual)\",\n
\"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"Hours
Studied\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 2,\n        \"min\": 2,\n        \"max\": 8,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          4,\n
8,\n          5\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Exam Score\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 11,\n        \"min\": 60,\n
\"max\": 90,\n        \"num_unique_values\": 5,\n        \"samples\":
[\n          70,\n          90,\n          75\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Attendance\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1,\n        \"min\": 7,\n        \"max\": 10,\n
\"num_unique_values\": 4,\n        \"samples\": [\n          9,\n
10,\n          8\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


```
Pearson Correlation Matrix (using pandas):
              Hours Studied  Exam Score  Attendance
Hours Studied       1.000000    0.997311    0.724407
Exam Score          0.997311    1.000000    0.736392
Attendance          0.724407    0.736392    1.000000

Pearson Correlation between 'Hours Studied' and 'Exam Score' (using
pandas):
0.9973108860238811

Pearson Correlation Coefficient (using scipy.stats.pearsonr):
0.997310886023881
P-value (using scipy.stats.pearsonr): 2.2804090962094822e-10

Pearson Correlation Coefficient (manual calculation):
0.997310886023881
```

4. What is multicollinearity? Does it affect simple regression?

Multicollinearity occurs in a multiple regression model when two or more independent variables are highly correlated with each other. In other words, one predictor variable can be linearly predicted from the others with a substantial degree of accuracy.

Example: If you are trying to predict house prices using both `square footage` and `number of rooms`, these two variables are often highly correlated (larger houses tend to have more rooms). This high correlation between predictors is multicollinearity.

Does it affect simple regression?

No, multicollinearity **does not directly affect simple linear regression**. Simple linear regression involves only one independent variable, so by definition, there are no *other* independent variables for it to be highly correlated with. Multicollinearity is a problem exclusively associated with **multiple linear regression**, where you have two or more independent variables.

In multiple regression, multicollinearity can lead to:

- **Unreliable coefficient estimates:** The coefficients of the correlated variables become unstable and difficult to interpret, as their individual effects are hard to distinguish.
- **Inflated standard errors:** This makes it harder to determine which independent variables are statistically significant.
- **Difficulty in interpreting the model:** It becomes challenging to understand the unique contribution of each predictor.

5. A dataset shows strong correlation but poor prediction accuracy. Why?

There are several reasons why a dataset might exhibit a strong correlation between variables but still result in poor prediction accuracy with a regression model:

1. **Non-linear Relationship:** Pearson correlation measures *linear* relationships. A strong correlation value might indicate a strong relationship, but if that relationship is non-linear (e.g., quadratic, exponential, S-shaped), a linear regression model will fail to capture it accurately, leading to poor predictions. For example, if Y increases with X up to a point and then decreases, the linear correlation might be low or moderate, but a non-linear model would predict well.

2. **Outliers:** Outliers can significantly influence the correlation coefficient, potentially making it appear stronger than the true underlying relationship for the majority of the data. If the model is sensitive to these outliers, it might fit them well but perform poorly on typical data points.

3. **Third Variable (Confounding Variable):** Two variables might appear strongly correlated because they are both influenced by a third, unobserved, or omitted variable. A simple regression model only considering the two correlated variables will miss the true driving force, leading to poor predictions when the confounding

variable changes. For example, ice cream sales and drowning incidents might be positively correlated, but both are caused by warm weather.

4. **Collinearity vs. Causation:** Correlation does not imply causation. While two variables might move together, one might not be causing the other, or the causal link might be indirect. A model built purely on correlation without understanding the underlying causal mechanisms can easily break down when conditions change.

5. **High Variance, Low Bias (Overfitting):** A model might perfectly capture the noise in the training data (leading to strong correlation in the training set) but fail to generalize to new, unseen data, resulting in poor prediction accuracy on test data. This is a classic case of overfitting.

6. **Limited Data Range or Quality:** If the strong correlation is observed only within a very limited range of the data, the model might not generalize well outside that range. Additionally, noisy data or measurement errors can lead to misleading correlations and poor predictive performance.

7. **Ignoring Other Relevant Predictors:** In simple linear regression, you only use one predictor. If the dependent variable is truly influenced by multiple factors, relying on a single, albeit strongly correlated, predictor will inevitably lead to poor prediction accuracy because the other influences are not accounted for.

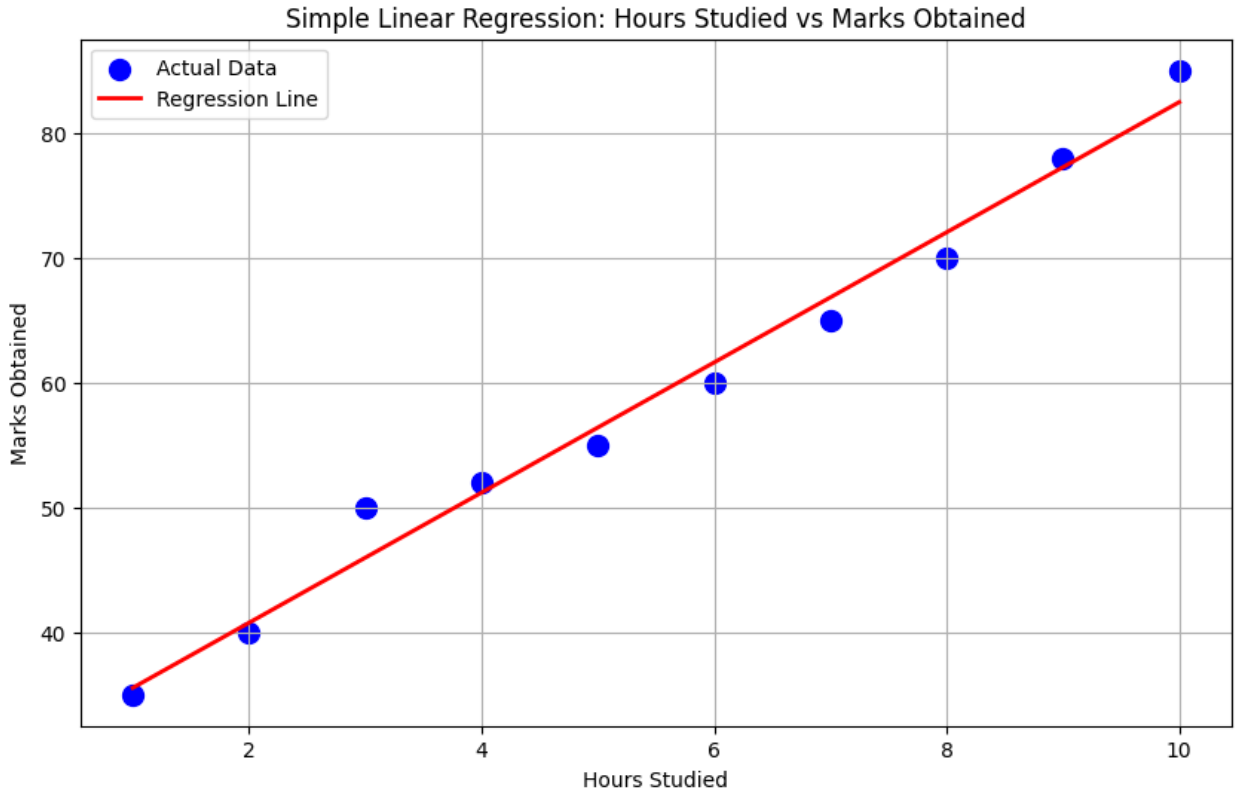
```
import numpy as np

# Sample data
X = [10, 20, 30, 40, 50]
Y = [15, 25, 35, 45, 55]

# Manual implementation of Pearson correlation
def pearson_coefficient(x, y):
    n = len(x)
    mean_x = np.mean(x)
    mean_y = np.mean(y)

    numerator = sum((x[i] - mean_x) * (y[i] - mean_y) for i in
range(n))
    denominator = (sum((x[i] - mean_x)**2 for i in range(n)) *
sum((y[i] - mean_y)**2 for i in range(n)))**0.5

    return numerator / denominator

# Calculate Karl Pearson's correlation coefficient
r = pearson_coefficient(X, Y)
print(f" Karl Pearson’s Correlation Coefficient (manual): {r:.4f}")

# Verification using scipy
from scipy.stats import pearsonr
```

```
r_scipy, _ = pearsonr(X, Y)
print(f" Verified using scipy: {r_scipy:.4f}")
```

```
 Karl Pearson’s Correlation Coefficient (manual): 1.0000
 Verified using scipy: 1.0000
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Step 1: Sample Data
data = {
    'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Marks_Obtained': [35, 40, 50, 52, 55, 60, 65, 70, 78, 85]
}
df = pd.DataFrame(data)

# Step 2: Split data into input (X) and output (y)
X = df[['Hours_Studied']]  # Independent variable
y = df['Marks_Obtained']   # Dependent variable

# Step 3: Train the Linear Regression Model
model = LinearRegression()
model.fit(X, y)

# Step 4: Get model parameters
slope = model.coef_[0]
intercept = model.intercept_
print(f"Regression Equation: Marks = {slope:.2f} * Hours +
{intercept:.2f}")

# Step 5: Predict values
y_pred = model.predict(X)

# Step 6: Evaluate the Model
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
print(f" Mean Squared Error (MSE): {mse:.2f}")
print(f" R² Score: {r2:.4f}")

# Step 7: Visualize the regression line
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='blue', label='Actual Data', s=100)
plt.plot(X, y_pred, color='red', linewidth=2, label='Regression Line')
plt.title('Simple Linear Regression: Hours Studied vs Marks Obtained')
plt.xlabel('Hours Studied')
plt.ylabel('Marks Obtained')
plt.grid(True)
```

```
plt.legend()
plt.show()

Regression Equation: Marks = 5.21 * Hours + 30.33
 Mean Squared Error (MSE): 3.68
 R² Score: 0.9839
```



Conclusion:Correlation measures the strength of a relationship, while regression is used to predict outcomes. Simple linear regression models a linear link between two variables. Tools like NumPy and Pandas help compute the Pearson correlation coefficient easily. A high correlation doesn't always mean good prediction due to factors like noise or nonlinearity.